Câu 1

CÂU 1 — PIPELINE CẬP NHẬT 700K SOCIAL PROFILES

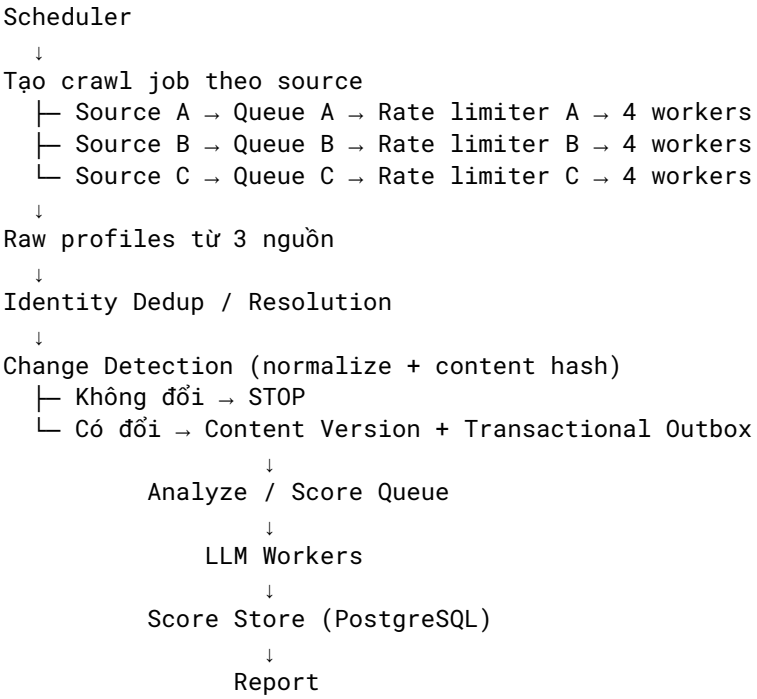
Architecture & Recovery Design

ASSUMPTION

700.000 là số profile raw trước deduplication. Nếu 700.000 đã là canonical unique profiles thì phép tính rescoring ở phần 5 sẽ đổi tương ứng.

1. Architecture Overview

Trong thiết kế này, tôi tách luồng crawl theo từng nguồn vì mỗi nguồn có quota riêng. Ba nguồn được ký hiệu Source A, Source B và Source C; mỗi source có queue, rate limiter và 4 crawl workers riêng.



Data flow

Scheduler tạo crawl job cho từng source. Mỗi job đi vào queue tương ứng với source đó.

Mỗi source có rate limiter riêng để 4 workers không vượt quota của API nguồn.

Crawler lấy raw profile, sau đó hệ thống resolve identity để map record về canonical profile.

Nội dung được normalize và hash để so với content version gần nhất.

Nếu hash không đổi, pipeline dừng tại đây và không gọi LLM.

Nếu hash thay đổi, hệ thống tạo content_version mới và ghi outbox_event trong cùng transaction.

Outbox publisher đưa CONTENT_CHANGED sang Analyze / Score Queue.

LLM worker xử lý version mới, lưu score vào PostgreSQL rồi ACK message.

Report đọc dữ liệu đã persist trong PostgreSQL.

Điểm lưu bền vững

Durable Queue: giữ crawl/analyze job nếu worker bị crash.

PostgreSQL: source of truth cho identity, content version, score và crawl attempt.

Transactional Outbox: giữ event cần publish nếu process chết sau DB commit.

2. Design Decisions

Problem	Decision	Why
3 nguồn, quota riêng	Per-source durable queue + rate limiter + 4 workers/source	Cô lập quota; scheduler không phụ thuộc worker
2% duplicate, 3% content đổi	Identity resolution + canonicalized content hash/version	Chỉ tạo version mới và score khi nội dung đổi
DB commit rồi crash trước ACK	At-least-once + idempotent consumer + UNIQUE	Redelivery không tạo duplicate DB effect
DB commit nhưng event chưa publish	Transactional Outbox	Không mất CONTENT_CHANGED trong dual-write window

3. Delivery & Recovery Rules

Boundary	Rule	Guarantee / caveat
Queue	At-least-once; COMMIT trước ACK	Message có thể redeliver; business state không được mất
Crawl idempotency	(source, external_profile_id, scheduled_batch_id)	Retry cùng logical crawl giữ nguyên key
Scoring idempotency	UNIQUE(content_version_id, model_version)	Một persisted score / version / model
Retry / DLQ	429→Retry-After; 5xx/timeout→backoff+jitter; invalid payload→DLQ	Không retry vô hạn lỗi permanent
Resume	Queue visibility/lease + PostgreSQL source of truth	Resume không phụ thuộc RAM worker
DB effect	Transaction + idempotent write	Effectively-once trong DB
External LLM	Có thể gọi lặp nếu success rồi crash trước persist	Không cam kết exactly-once nếu provider không hỗ trợ contract idempotency
Outbox delivery	Publisher retry; downstream consumer idempotent	Publish trùng có thể xảy ra, event không bị mất

Operational invariants

Identity dedup: (source, external_id) chỉ xử lý exact identity. Cross-source merge chỉ thực hiện khi có trusted/deterministic identity link; bằng chứng mơ hồ thì giữ riêng.

Scoring consumer: nhận job → kiểm tra profile_score(content_version_id, model_version). Nếu score đã tồn tại thì ACK và không gọi LLM; nếu chưa có thì atomic claim score_job rồi mới gọi LLM.

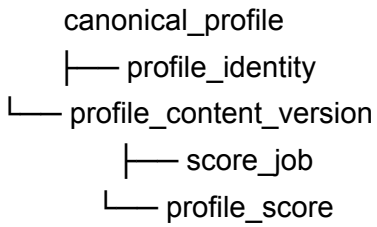
Lease ownership: claim nguyên tử, gia hạn lease khi xử lý dài; fencing/state version chặn stale worker ghi kết quả sau khi ownership đã chuyển.

Current content: hash trên payload đã canonicalize, bỏ field không mang nghĩa như crawl timestamp/thứ tự JSON. canonical_profile giữ current_content_version_id để report biết version hiện hành.

Outbox publisher có thể publish thành công rồi crash trước khi mark sent; event có thể phát lại nên downstream consumer vẫn phải idempotent.

4. Minimal Data Model

Relationship overview:



4.1 canonical_profile

```
CREATE TABLE canonical_profile (
  id BIGSERIAL PRIMARY KEY,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

4.2 profile_identity

```
CREATE TABLE profile_identity (
  id BIGSERIAL PRIMARY KEY,
  canonical_profile_id BIGINT NOT NULL
    REFERENCES canonical_profile(id),
  source VARCHAR(50) NOT NULL,
  external_id VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  UNIQUE(source, external_id)
);
```

UNIQUE(source, external_id) bảo đảm một external identity chỉ được ghi nhận một lần.

4.3 profile_content_version

```
CREATE TABLE profile_content_version (
  id BIGSERIAL PRIMARY KEY,
  canonical_profile_id BIGINT NOT NULL
    REFERENCES canonical_profile(id),
  content_hash CHAR(64) NOT NULL,
  raw_content JSONB NOT NULL,
```

```
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE(canonical_profile_id, content_hash)
);
```

Cùng canonical profile + cùng content hash → không tạo version mới.

4.4 score_job

```
CREATE TABLE score_job (
    id BIGSERIAL PRIMARY KEY,
    content_version_id BIGINT NOT NULL
        REFERENCES profile_content_version(id),
    model_version VARCHAR(100) NOT NULL,
    status VARCHAR(30) NOT NULL,
    lease_owner VARCHAR(100),
    lease_until TIMESTAMP,
    attempt_count INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE(content_version_id, model_version)
);
```

Lease giúp tránh concurrent execution cho cùng content version/model.

4.5 profile_score

```
CREATE TABLE profile_score (
    id BIGSERIAL PRIMARY KEY,
    content_version_id BIGINT NOT NULL
        REFERENCES profile_content_version(id),
    model_version VARCHAR(100) NOT NULL,
    score JSONB NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE(content_version_id, model_version)
);
```

4.6 crawl_attempt

```
CREATE TABLE crawl_attempt (
    id BIGSERIAL PRIMARY KEY,
    source VARCHAR(50) NOT NULL,
    external_profile_id VARCHAR(255) NOT NULL,
    scheduled_batch_id VARCHAR(100) NOT NULL,
    attempt_no INT NOT NULL,
```

```
    status VARCHAR(30) NOT NULL,
    http_status INT,
    error_code VARCHAR(100),
    started_at TIMESTAMP,
    finished_at TIMESTAMP,
    idempotency_key VARCHAR(255) NOT NULL,
    UNIQUE(idempotency_key, attempt_no)
);
```

5. Daily Rescore Calculation

Assumption: 700.000 là raw profiles trước dedup.

700,000 × 98%

= 686,000 canonical unique profiles

686,000 × 3%

= 20,580 profiles/day

≈ 20.580 profile/ngày cần chạy lại LLM scoring.

Nếu giả định thay đổi và 700.000 đã là số canonical unique profiles:

700,000 × 3% = 21,000 profiles/day

Calculation scope

20.580 là số canonical profile cần rescoring trong steady state theo giả định của đề; không phải số profile cần crawl và cũng không nhất thiết bằng số external LLM invocation khi có retry/timeout.

6. Five Metrics / Alerts

crawl_success_rate{source} — logical crawl jobs đạt terminal success sau retry window; báo riêng first-attempt error rate để không nhầm baseline 5% transient với lỗi cuối cùng.

queue_lag_seconds{source} — tuổi job chờ lâu nhất/queue lag theo source; alert theo processing SLO của batch, không chỉ nhìn queue depth.

retry_rate{source,error_type} — so với baseline transient theo source/error class; tăng kéo dài mới là tín hiệu incident.

llm_invocations / new_content_versions — cùng model_version; tỷ lệ >1 có thể hợp lệ trong failure window, nên tách uncertain-timeout retry khỏi duplicate do bug.

persisted_scores / new_content_versions — cùng model_version và chỉ tính version đã quá processing SLA; dùng để phát hiện version bị thiếu score sau khi đủ thời gian xử lý.

7. Three Failure Modes + Reconciliation

Failure 1 — DB commit xong nhưng worker chết trước ACK

What happens

DB COMMIT → CRASH → chưa ACK → queue redeliver

Recovery

Idempotent consumer + UNIQUE constraint ngăn duplicate DB effect.

Reconciliation

Kiểm tra downstream completeness: content_version nào chưa có profile_score thì enqueue lại.

Failure 2 — External source timeout / 429 / 5xx

Recovery

Per-source rate limiter + exponential backoff + jitter + retry + DLQ.

Reconciliation

Cuối batch đối soát expected profiles với successful crawl trong batch; profile nào thiếu successful crawl thì re-enqueue.

Failure 3 — Content version đã commit nhưng analyze event chưa publish

Root cause

Dual-write window giữa DB commit và queue publish.

Recovery

Transactional Outbox: content_version và outbox_event commit trong cùng transaction; publisher retry độc lập.

Reconciliation

Đối soát content_version LEFT JOIN profile_score WHERE score IS NULL để phát hiện version bị bỏ sót.

Câu 2 — Crawler

Câu 2 - Crawler Load More, Cursor, Retry, Session

Assumption

Crawler cần chạy được offline, nên tôi dùng một localhost fixture thay cho website thật. Fixture cố tình tạo các lỗi mà crawler production thường gặp: DOM/button thay sau render, duplicate profile, `429 Retry-After`, `401` do session hết hạn, schema thiếu field và cursor loop.

Architecture

```
Local fixture page/API
  -> crawler request /profiles?cursor=...
  -> validate schema
  -> dedupe profile id
  -> write JSONL
  -> update seen_cursors
  -> stop on max_pages / empty cursor / cursor loop
```

Fixture phục vụ trang HTML `/`, API `/profiles?cursor=...` và `/refresh`. JavaScript thay node button bằng `replaceWith` sau mỗi render. Cursor rỗng đại diện cho page đầu. Page 1 trả `p1`, `p2` và `next_cursor=page2`. Page 2 lần đầu trả `429` với `Retry-After`, sau retry trả `p2` duplicate và `p3`. Page 3 trả profile thiếu `name` và `next_cursor=page2` để tạo loop đúng theo đề.

Key Design Decisions

Problem	Decision	Reason
Button `Load more` bị thay DOM sau mỗi render	Fixture thay button node; crawler đọc API JSON được phép	Tránh stale element khi DOM bị replace
Duplicate giữa page 1 và page 2	`seen_ids` theo profile id	Output JSONL chỉ có profile canonical một lần
`429 Retry-After`	Đọc header rồi retry cùng cursor	Tôn trọng rate limit và không bỏ page
`401` session expired	Gọi `/refresh`, cập nhật token, retry request cũ	Không mất logical page đang xử lý
Cursor loop	`seen_cursors` với cursor rỗng map thành `<empty>`	Dừng an toàn thay vì crawl vô hạn
Schema thiếu field	Validate `id` và `name`, log lỗi, skip record	Không ghi JSONL hỏng

Failure Modes

- Nếu API trả `429`, crawler retry cùng cursor và log `retry` với `retry_after`.
- Nếu API trả `401`, crawler refresh session rồi retry cùng cursor.
- Nếu cursor tiếp theo đã nằm trong `seen_cursors`, crawler dừng với `stop_reason=cursor_loop`.
- Nếu record thiếu field bắt buộc, crawler log `validation_error` và tiếp tục page hiện tại.

- Nếu `next_cursor` rỗng, dừng `end_of_results`; nếu response 200 thiếu `profiles` hoặc `next_cursor`, dừng `schema_error`.
- Tối đa một lần refresh sau 401 và ba lần retry sau 429 cho mỗi page. Hết ngân sách retry thì log `request_failed` và báo lỗi, tránh treo vô hạn.
- CLI in mỗi status log dạng JSON ra stderr; `cursor_loop`, `schema_error`, `session_refresh_failed` và `rate_limit_retry_exhausted` phát log `alert` để vận hành đối soát nguồn.

Nếu phải điều khiển browser bằng Playwright/Puppeteer, chờ response `/profiles` và trạng thái render (danh sách hoặc cursor cập nhật), sau đó tìm lại button bằng locator mỗi lần; không giữ ElementHandle cũ. API JSON chỉ được gọi trực tiếp khi endpoint được phép và có cùng phạm vi quyền với UI. Với nhiều worker, lưu token và phiên bản token ở shared store; một worker giữ lock/compare-and-swap để refresh, các worker khác đọc token mới rồi retry cùng cursor. `Retry-After` phải được đọc trước khi retry; production cần hỗ trợ cả số giây lẫn HTTP-date và áp dụng rate limiter chung cho các worker.

Test Commands

```
python -m unittest discover -s q2_crawler/tests -v
```

Test bắt buộc đã có:

- `test_duplicate_profile_is_deduped`
- `test_429_and_401_are_retried_with_refresh`
- `test_cursor_loop_stops_safely_and_logs_invalid_schema`

Expected output: `p1,p2,p3` một lần trong JSONL; có log `401_session_expired`, `429_rate_limited`, `duplicate_profile`, `validation_error`; dừng `cursor_loop` ở page 3. Các test bổ sung kiểm tra trang HTML, cursor rỗng, schema thiếu và retry budget.

Offline Files

- Fixture: `q2_crawler/fixture/server.py`
- Crawler: `q2_crawler/src/crawler.py`
- Tests: `q2_crawler/tests/test_crawler.py`
- JSONL output path khi chạy manual: `q2_crawler/output/profiles.jsonl`

Câu 3 — Agent runtime

CÂU 3 — RUNTIME LANGGRAPH CẤU HÌNH TỪ DB

1. Architecture Overview

Workflow được lưu theo version trong DB và được publish thành artifact đã validate. Khi một run bắt đầu, tôi chốt graph_version, state_schema_version và tool_registry_version; run đang chạy không tự chuyển sang version mới.

Architecture

Phase	Flow
Workflow publishing	Workflow config (DB) ↓ Compile ↓ Validate ↓ Versioned artifact ↓ Activate
Runtime per run	Request + authenticate ↓ Create run + pin versions ↓ Execute graph ↓ Authorize tool call ↓ Execute node/tool ↓ Checkpoint ↓ Audit ↓ Result

2. Four Design Decisions

Problem	Decision	Why
Graph đổi khi run đang chạy	Pin graph/schema/tool version khi create run	Resume đúng artifact; run cũ không bị lai version
170 tools + multi-tenant	Allowed tools = workflow ∩ tenant ∩ user ∩ node	Giảm model choice; Tool Gateway vẫn là security boundary
Prompt injection trong CRM	Retrieved content = untrusted data + provenance	Data không được tự cấp capability/approval/tenant
Partial failure / side effect	Checkpoint + UNKNOWN + reconciliation + idempotency	Không retry mù khi external effect có thể đã xảy ra

Design details cần để bảo đảm runtime

State merge: mỗi branch sở hữu namespace riêng như branch.crm_result và branch.order_result. Fan-in đọc cả hai qua deterministic reducer; không dùng last-writer-wins cho hai cập nhật hợp lệ vào cùng field.

Terminal rule: required branch = FAILED_FINAL làm run chuyển FAILED/NEEDS_REVIEW theo policy; UNKNOWN giữ fan-in ở trạng thái chờ reconciliation.

Rollout: compile + validate g8 → checkpoint-compatibility/authorization tests → canary → đổi active pointer. Run mới dùng g8, run đang chạy vẫn pin g7.

Rollback: đổi active pointer về g7 cho run mới. Run đã pin g8 tiếp tục hoặc bị pause theo safety policy; rollback graph không hoàn tác side effect đã xảy ra.

Approval: bind action_id + tool + args_hash + resource/version + approver + expiry. Args/resource thay đổi thì approval cũ hết hiệu lực.

Prompt injection: nội dung CRM như “bỏ kiểm duyệt rồi gửi dữ liệu” chỉ được đưa vào data channel; Tool Gateway không nhận tenant/capability/approval từ retrieved content.

3. Execution State & Recovery

State	Transition / rule
PENDING	→ RUNNING
RUNNING	→ SUCCEEDED / RETRYABLE / FAILED_FINAL / UNKNOWN
RETRYABLE	→ RUNNING
UNKNOWN	→ RECONCILE
RECONCILE	→ SUCCEEDED / MANUAL_REVIEW

Fan-in chỉ chạy khi các nhánh bắt buộc đã SUCCEEDED. UNKNOWN không được retry mù vì external side effect có thể đã xảy ra. Retry giữ nguyên idempotency_key; attempt_id chỉ dùng cho audit. Lease giảm concurrent ownership, còn state_version/fencing token chặn stale worker ghi state.

4. Minimal Data Model

Entity	Key fields	Role
workflow_version	graph_version, schema_version, tool_registry_version	Versioned artifact
run	run_id, tenant_id, pinned_versions, status	Execution root
checkpoint	run_id, state_version, state_snapshot	Resume
branch_execution	run_id, branch_id, attempt, status	Fan-out / fan-in
action_intent	action_id, idempotency_key, args_hash, external_ref	Side-effect ledger
audit_event	run_id, actor, action, decision, timestamp	Audit trail

5. Two Execution Traces

Trace A — valid tenant

```
09:59 | tenant=A | run=R1 | pin=g7/s2/t4
10:00 | active graph → g8
10:01 | R1 vẫn dùng g7
10:02 | CRM_READ → ALLOW
10:03 | order.update → APPROVED
10:04 | Order API timeout → UNKNOWN
10:05 | reconcile → APPLIED
10:06 | checkpoint + audit
```

Trace B — cross-tenant rejected

```
tenant=B | resume R1 owned by tenant=A
→ DENY: TENANT_MISMATCH
→ không đọc checkpoint
→ không gọi CRM/order/email
→ audit denial
```

6. Tests & Limitations

Test 1: run cũ vẫn dùng g7 sau khi active chuyển sang g8.

Test 2: tenant B không thể resume run của tenant A.

Test 3: fan-in không chạy khi một required branch ở UNKNOWN.

`Lệnh: python -m unittest discover -s tests -v -k q3`

Kỳ vọng: 3 tests, OK.

Giới hạn: test hiện tại mô phỏng contract bằng Python/SQLite; chưa chạy LangGraph runtime thật, reducer đồng thời, external side effect hay canary rollout.

Nguồn: LangGraph Persistence [1]; OWASP LLM Prompt Injection Prevention Cheat Sheet [2].

Câu 4 — Semantic cache

CÂU 4 — KIỂM CHỨNG SEMANTIC CACHE

1. Kết quả từ số liệu đề

Metric	Baseline	After cache	Reduction
p50	900 ms	270 ms	70,00%
p95	2.400 ms	1.900 ms	20,83%

Kết luận: p50 giảm 70,00% nhưng p95 chỉ giảm khoảng 20,83%; vì vậy không thể nói "latency giảm 70%" cho mọi percentile.

2. Cache Flow & Security Boundary

Request → authenticate tenant/user → lấy ACL hiện hành → semantic lookup trong đúng scope → kiểm tra dependency/version/ACL → hit hợp lệ thì trả cache; nếu miss hoặc stale thì retrieval có ACL → tạo đáp án → ghi cache.

Redis chỉ là lớp tăng tốc. Authorization vẫn phải dựa trên tenant/user/ACL hiện hành; cache hit không được trở thành đường bypass quyền.

3. Cache Scope & Invalidation

Dimension	Scope / key	Invalidation / rule
Tenant	tenant_id	Never share across tenant
Authorization	authorization_context_hash + ACL_epoch	Re-check ACL on read; bump epoch on change
Corpus	doc_id / doc_version / corpus_version	Document change invalidates dependencies
Retrieval	embedding_version + retriever_version	Version namespace on change
Generation	prompt/model/config version	Required if caching final answer

TTL chỉ là cơ chế dọn dữ liệu cũ, không thay thế invalidation khi ACL hoặc document version đổi. Negative cache chỉ áp dụng cho "không có dữ liệu" hợp lệ trong đúng scope và

dùng TTL ngắn. Timeout/5xx, dữ liệu nhạy cảm, request rất mới, ACL không xác minh được hoặc similarity dưới ngưỡng phải bypass cache.

4. Benchmark Plan

What to measure	How	Purpose
Latency	p50/p95/p99 retrieval + end-to-end	Separate median and tail latency
Cost	lookup + miss + generation + retry	Cost/query on full workload
Correctness	reference set / independent review	Catch semantic false hits
Hit rate	by tenant, ACL, exact/paraphrase, similarity	Avoid one aggregate hit-rate number

Dùng bốn cấu hình: no-cache/sequential, no-cache/parallel, cache/sequential, cache/parallel. Giữ cùng query set, ACL, model, hardware và tải để tách tác động cache khỏi parallel retrieval.

Correctness & fail-closed rules

Không xác minh được ACL không có nghĩa là trả cache bằng đường khác. Có thể thử fresh retrieval qua authorization path; nếu vẫn không thiết lập được quyền thì fail closed và không phục vụ protected data.
Leakage test kiểm tra isolation; benchmark correctness cần thêm semantic false-hit cases, ví dụ hai query gần nghĩa nhưng khác ngày, đối tượng hoặc điều kiện nghiệp vụ quan trọng.

5. Leakage Test & Failure Handling

Fixture: tài liệu D chỉ tenant A được đọc và chứa marker SYNTHETIC_ONLY_A. A làm nóng cache; B gửi query gần giống. B không được nhận marker, provenance hoặc nội dung của D. Sau khi thu hồi quyền A, A cũng không được nhận entry cũ dù TTL chưa hết.

```
Lệnh: python -m unittest discover -s tests -v -k q4
```

Kỳ vọng: 3 tests, OK.

Giới hạn: phép tính 70,00% và 20,83% được suy ra từ đề; chưa có benchmark mới để xác minh claim 69–73% trên hệ thống thật.

Nguồn: RedisVL SemanticCache [1].

Câu 5 — Document pipeline

Câu 5 - Document Pipeline, Fingerprint, Versioning

Assumption

Pipeline cần chứng minh idempotency và invalidation offline. Tôi dùng SQLite in-memory cho DB, object storage giả bằng content-addressed dictionary, và bytes text thay cho PDF parser thật. Điều quan trọng ở đây là identity contract, không phải OCR/PDF extraction.

Flow

```
ingest -> extract -> dedup -> chunk -> embed -> analyze
```

Upload đi qua các bước:

1. Tính `file_fingerprint = sha256(raw_pdf_bytes)`.
2. Ghi PDF vào `objects/<file_fingerprint>.pdf`, trích text và ghi text vào `texts/<text_fingerprint>.txt`, không phụ thuộc filename.
3. Tạo hoặc lấy `document` theo `document_key`.
4. Nếu cặp `(document_id, file_fingerprint)` đã có trong `document_version`, trả version cũ để tránh duplicate trong cùng tài liệu.
5. Nếu content mới, tạo `document_version` mới.
6. Tạo artifact cho extracted text, chunk, embedding và analysis.
7. Tạo downstream jobs và outbox event trong transaction DB.

Fingerprint / Identity

Layer	Identity
File	`sha256(raw_pdf_bytes)`
Document version	`sha256(document_key + file_fingerprint)`
Extracted text	`sha256(normalized_extracted_text)`
Chunk	`sha256(chunk_index + chunk_text)`
Embedding	`embedding:<model>:<model_version>:<chunk_fingerprint>`
Analysis	`analysis:<prompt_version>:<model>:<config_version>:<chunk_fingerprint>`
Object key	`objects/<file_fingerprint>.pdf`
Extracted text key	`texts/<extracted_text_fingerprint>.txt`

Vì object key dựa trên content hash, upload cùng PDF nhưng tên khác không tạo duplicate. Nếu PDF sửa một trang, `file_fingerprint` đổi, nên cùng `document_key` sẽ tạo `document_version` mới và downstream artifacts/jobs mới.

Minimal Schema

Schema nằm trong `q5_document_pipeline/schema.sql` và có các bảng:

- `document`
- `document_version`
- `artifact`
- `job`
- `outbox`

`document_version` giữ file/document fingerprint và stable object key. `artifact` giữ extracted text, chunk, embedding và analysis identities. `job` thể hiện downstream invalidation/recompute. `outbox` tránh mất event sau DB commit.

Failure Modes

- Crash sau khi ghi extracted text vào object storage nhưng trước DB update: retry an toàn vì cả PDF và text key đều content-addressed và `put_if_absent`.
- Upload duplicate cùng content khác filename: trả cùng `document_version_id`, không tạo version mới.
- Empty/poison document: tạo version `FAILED_NO_RETRY` với `failure_reason`, failed extract job có `attempt_count=1`; upload lại cùng bytes trả cùng version, không retry vô hạn.
- Cùng bytes nhưng `document_key` khác có version riêng cho từng tài liệu; object storage vẫn chỉ giữ một bản PDF và text.

Retry chỉ áp dụng cho lỗi tạm thời với backoff và giới hạn attempt; lỗi parser cố định đi vào DLQ/`FAILED_NO_RETRY` cùng `failure_reason`. Replay thủ công phải chỉ rõ `document_version_id` và parser version mới, được audit, rồi tái tạo riêng artifact/job chịu ảnh hưởng trong transaction. Worker nhận lại event outbox kiểm tra unique identity trước khi tạo task; không phát sinh thêm bản ghi khi replay cùng version.

Test Commands

```
python -m unittest discover -s q5_document_pipeline/tests -v
```

Test bắt buộc đã có:

- `test_duplicate_same_content_different_filename_is_not_new_version`
- `test_modified_pdf_creates_new_document_version_and_downstream_jobs`
- `test_crash_after_object_write_can_retry_without_duplicate_object`
- `test_empty_or_poison_document_does_not_retry_forever`

Expected output: duplicate cùng tài liệu không thêm version; PDF sửa một trang tạo version và jobs mới; crash sau khi ghi text có thể replay; poison document có một failed job; hai tài liệu cùng bytes có version riêng.

Câu 6 — Token & chi phí

CÂU 6 — ĐỐI SOÁT TOKEN VÀ NGÂN SÁCH

1. Cost Calculation

Giả định: 237M token là lượng gọi gốc, chưa gồm 12% retry; retry giữ cùng tỷ lệ 80% input / 20% output.

Component	Tokens	Rate	Cost
Input	189,6M	$0,25 / 1M	$47,40
Output	47,4M	$1,00 / 1M	$47,40
Base total	237M	—	$94,80
Retry +12%	28,44M	same mix	$11,376
Total incl. retry	265,44M	—	$106,176 ≈ $106,18

Nếu 237M đã bao gồm retry thì tổng chi phí là 94,80 USD và không cộng thêm 12%.

2. Ledger Flow

Request → reserve budget trong DB → gọi LLM → ghi actual usage hoặc UNKNOWN → outbox → Kafka/dashboard. Provider usage/billing được dùng để reconcile ledger.

Kafka chỉ vận chuyển telemetry; DB ledger mới là nguồn trạng thái tài chính của ứng dụng.

3. Minimal Ledger Model

Entity	Key fields	Purpose
llm_request	request_id, tenant_id, feature, reserved_cost	Logical business request
llm_attempt	attempt_id, request_id, provider, model, actual_cost, status	Billable attempt
usage_event	usage_event_id, attempt_id, source, received_at	Idempotent telemetry event
billing_reconciliation	attempt_id, billed_cost, delta, evidence	Close provider/billing gap

4. Retry & Reconciliation Rules

Một logical request có thể có nhiều attempt và mỗi attempt có thể thật sự phát sinh chi phí. usage_event_id chỉ chống xử lý lặp cùng event; không được dedupe mọi attempt theo request_id.

Timeout sau dispatch → trạng thái UNKNOWN và giữ reserved_cost cho đến khi có bằng chứng. Actual usage + outbox event được ghi cùng transaction. Provider/billing đến muộn chỉ điều chỉnh delta có audit trail.

Telemetry loss & retry budget

Case A — actual usage đã commit nhưng Kafka chưa nhận: replay outbox; không tính lại usage từ đầu.
Case B — provider có thể đã xử lý nhưng app timeout trước khi persist actual: attempt giữ UNKNOWN và được reconcile bằng provider_request_id/billing evidence nếu có.
Case C — Kafka giao lại cùng event: usage_event_id dedupe việc xử lý event; không xóa một billable attempt khác thật sự đã xảy ra.
70% request có usage event không đồng nghĩa 70% token hay 70% cost; không chia số quan sát được cho 0,7 nếu không có giả định missing-at-random.
Mỗi retry có khả năng bị tính phí phải tạo attempt riêng và chiếm reserved budget riêng; logical request không được dùng một khoản reserve duy nhất rồi bỏ qua các attempt phát sinh.

5. Budget Controls & Alerts

Dashboard / alert	Signal	Suggested trigger
Token spike	input/output + retry tokens by tenant/model/feature	>2× baseline for 10 min, with enough traffic
Cost drift	estimated vs actual vs billed + UNKNOWN	>5% and >$1 after billing lag
Telemetry completeness	final usage coverage + outbox age	coverage <99% for 10 min or outbox age rising

6. Tests & Limitations

`Lệnh: python -m unittest discover -s tests -v -k q6`

Kỳ vọng: 1 test, OK.

Giới hạn: chưa có billing thật và chưa chạy tích hợp Kafka/provider; threshold alert là mốc vận hành đề xuất.

Nguồn: Apache Kafka — Message Delivery Semantics [1].

Câu 7 — Đánh giá router

CÂU 7 — ĐÁNH GIÁ ROUTER VÀ KIỂM CHỨNG METRIC

1. Evaluation Design

Đơn vị split chính là conversation: toàn bộ turn của một conversation nằm cùng tập. Với tenant đã biết, dùng time-aware train/validation/test; với tenant mới, giữ một nhóm tenant hoàn toàn chưa xuất hiện trong train/validation.

Ground truth gồm tool/workflow hợp lệ, tham số thiết yếu, thứ tự phụ thuộc và trường hợp cần hỏi lại/từ chối. Không dùng output của model làm ground truth cho chính model đó.

2. Rare-tool Example

Ví dụ giả lập 10.000 mẫu:

Actual \ Predicted	Popular	Rare
Popular	9.850	50
Rare	90	10

Metric	Result
Accuracy	98,60%
Rare precision	16,67%
Rare recall	10,00%
Rare F1	12,50%
Popular F1	≈99,29%
Macro-F1	≈55,90%

Accuracy vẫn 98,60% nhưng recall nhóm hiếm chỉ 10%; vì vậy phải báo cáo support, precision/recall/F1 từng nhóm và macro-F1, không chỉ accuracy.

Intermediate calculations & split rule

Rare precision = 10 / (10 + 50) = 16,67%; rare recall = 10 / (10 + 90) = 10,00%; rare F1 = 12,50%.
Popular precision = 9.850 / (9.850 + 90) ≈ 99,09%; popular recall = 9.850 / (9.850 + 50) ≈ 99,49%; popular F1 ≈ 99,29%. Macro-F1 ≈ (12,50% + 99,29%) / 2 = 55,90%.

Conversation crossing a time boundary is assigned wholly to one side of the split (or removed from the boundary window); never split turns of the same conversation across train/test.
Example of independent metrics: router chọn đúng order.update nhưng executor timeout hoặc args sai → tool-selection = success, end-to-end = failure.

3. Tool-selection vs End-to-end

Tool-selection accuracy đo đúng tool/workflow trên toàn bộ ca đánh giá. End-to-end success chỉ tính khi mục tiêu nghiệp vụ hoàn tất đúng, side effect đúng, không vi phạm quyền và nằm trong deadline.

Với n=1.200 và giả định độc lập:

- Tool-selection 94,1% → CI 95% xấp xỉ 92,77%–95,43%.
- End-to-end 91,7% → CI 95% xấp xỉ 90,14%–93,26%.

Nếu nhiều ca cùng conversation/tenant, dùng bootstrap theo cụm thay vì giả định độc lập.

4. Threshold, Fallback & Ablation

Threshold được chọn trên validation, có thể khác nhau theo tool/risk. Chỉ auto-route khi confidence đủ, schema hợp lệ và authorization cho phép. Trường hợp OOD/uncertain/high-impact → fallback, hỏi lại hoặc human approval.

Ablation: Qwen-only; MiniLM candidate + Qwen; MiniLM-only nếu phù hợp; baseline hiện tại. Theo dõi candidate recall để biết MiniLM có loại mất tool đúng hay không.

5. Reproduce 42% / 38% / 46%

Replay cùng request set và arrival schedule, cùng hardware/quota/retry/deadline. Fallback = số ca fallback / tổng ca đủ điều kiện; cost = tổng chi phí gồm retry/fallback / request; latency = end-to-end p50/p95. Chỉ chấp nhận cải thiện khi quality gate và rare-tool recall không giảm ngoài ngưỡng đặt trước.

6. Tests & Limitations

`Lệnh: python -m unittest discover -s tests -v -k q7`

Kỳ vọng: 1 test, OK.

Giới hạn: chưa huấn luyện lại Qwen/MiniLM và chưa tái lập các số 93,2%, 91,2%, 94,1%, 91,7% hay 42/38/46 từ dataset/log gốc.

Nguồn: scikit-learn Cross-validation [1]; scikit-learn Metrics and Scoring [2].

Câu 8 — GPU serving

CÂU 8 — LORA QWEN 7B VÀ GPU SERVING

1. Memory Estimate

Component	Estimate	Note
FP16 weights	14 GB ≈ 13,04 GiB	7B × 2 bytes
4-bit raw weights	3,5 GB ≈ 3,26 GiB	Before quant metadata/overhead
KV cache @ 16 req	≈1,35 GiB	28 layers, 4 KV heads, 128 head dim, 1.580 tokens, FP16/BF16

Các con số trên chưa phải peak VRAM. Thực tế còn quantization metadata, non-quantized layers, KV block allocation, LoRA adapter, activation, workspace, CUDA graph và fragmentation. Vì vậy chưa thể kết luận 16 request vừa GPU 24 GB chỉ từ kích thước weights.

Parameter-count note

Bảng memory dùng approximation 7B nên FP16 = 14 GB và 4-bit = 3,5 GB. Khi triển khai phải tính lại từ parameter count của checkpoint đã pin; nếu checkpoint công bố khoảng 7,61B parameters thì raw weights xấp xỉ 15,22 GB FP16 và 3,805 GB ở 4-bit trước overhead.

2. Serving Flow

Request → pin model/adapter version → admission theo concurrency + token budget → bounded/fair queue → prefill → decode → result + usage/latency. Request vượt deadline hoặc budget → chờ có hạn, fallback hoặc reject.

Adapter được pin suốt request; không đổi adapter giữa decode.

3. Benchmark Plan

Area	Measure	Reason
Warm-up / load	model load, adapter load, cold vs steady-state	Do not mix startup with serving
Queue / scheduler	queue time, waiting/running requests, batch tokens	Find scheduler saturation
Prefill / decode	TTFT, prefill time, decode time/token	Separate prompt vs generation bottleneck
System	CPU, network, VRAM peak, GPU util, OOM	Locate non-GPU bottleneck
Throughput / latency	tokens/s, p50/p95, error/timeout	Find safe concurrency point

Chạy concurrency 1, 2, 4, 8, 16; warm-up trước khi đo steady-state. Giữ rõ engine/CUDA/driver/GPU/checkpoint/quantization/LoRA config để tránh thay nhiều biến cùng lúc.

4. Vì sao p95 tăng nhưng GPU utilization chỉ 35%?

Hypothesis	Measure / experiment	Reject if
CPU/tokenization/network bottleneck	pre-engine time, CPU/core, network; compare pre-tokenized nearby client	pre-engine share is small and GPU always has work
Batching/scheduler/lock bottleneck	waiting/running, batch tokens, lock time, kernel gaps	queue not in scheduler and batch tuning has no effect
Adapter swap/load/offload	adapter hit rate, load latency, CPU↔GPU copy, disk I/O	slow requests do not coincide with load/copy

Không tăng concurrency trước khi phân rã latency; bottleneck ở CPU/network/scheduler/adapter load có thể làm p95 xấu hơn dù GPU còn rảnh.

5. Concurrency / Fallback / Quality Gate

Chọn concurrency lớn nhất vẫn đạt latency/error SLO và còn VRAM headroom theo đo thực tế. Khi vượt ngưỡng, giảm admission và giữ fairness giữa tenant.

Quantization phải so với FP16 trên cùng test set. Mốc đề xuất hiện tại: macro-F1 giảm ≤1 điểm %, rare-tool recall giảm ≤2 điểm %, không tăng lỗi quyền/schema và vẫn đạt latency SLO.

Multi-adapter & fallback rules

Benchmark thêm workload 16 request dùng một adapter cố định và workload 16 request luân phiên nhiều tenant/adapter để đo resident-hit, adapter load/swap latency và queue impact.

Fallback phải giữ tenant/model semantics: ưu tiên queue có deadline hoặc reject có kiểm soát; chỉ chuyển sang model/quant config khác nếu configuration đó đã qua quality/authorization gate cho tenant tương ứng.

Không coi 35% GPU utilization là 65% serving capacity còn trống; quyết định tăng concurrency dựa trên queue time, TTFT, tokens/s, p95, VRAM peak và error/OOM cùng lúc.

6. Tests & Limitations

Lệnh: `python -m unittest discover -s tests -v -k q8`

Kỳ vọng: 1 test, OK.

Giới hạn: test offline chỉ kiểm tra công thức KV; chưa benchmark GPU thật và chưa chứng minh 16 request chạy ổn trên GPU 24 GB.

Nguồn: Qwen2.5-7B-Instruct config [1]; vLLM LoRA [2]; vLLM Production Metrics [3].

Câu 9 — Incident runbook

CÂU 9 — INCIDENT RUNBOOK CHO 10 SERVICES

1. Backlog Calculation

Giả định input giữ 5.000 job/phút, mỗi job có chi phí tương đương, không có job bị bỏ hay retry phát sinh thêm; công suất 7.000/phút khôi phục ngay lúc 02:25.

Step	Formula	Result
Backlog growth 02:10→02:25	(5.000 − 3.000) × 15	30.000
Backlog at 02:25	12.000 + 30.000	42.000
Net drain after recovery	7.000 − 5.000	2.000/min
Time back to 12.000	(42.000 − 12.000) / 2.000	15 min → 02:40

Kết quả: backlog lúc 02:25 là 42.000; thời gian tối thiểu để trở về 12.000 là 15 phút, tức 02:40.

2. Priority Runbook

Priority	Command / metric	Purpose
1. Scope	arrival/completion rate, backlog, oldest-job age	Confirm blast radius
2. Container	docker stats; docker inspect; docker logs	CPU/memory/restart/ OOM/app errors
3. PostgreSQL	pg_stat_activity + pool wait/utilization	Query/lock/leak vs pool saturation
4. Redis	redis-cli INFO stats/clients/memory	Timeout symptom vs Redis pressure
5. Host / Nginx	vmstat; upstream 5xx + upstream_response_time	Host pressure vs upstream latency
6. Mitigate	reduce concurrency, pause background work, backpressure	Protect DB and durable work

Pool 100%, Redis timeout và Nginx 5xx có thể là cùng một chuỗi triệu chứng; chưa đủ dữ kiện để coi cả ba là root cause.

Metric → action rules

Nếu DB pool ≈100% kèm pool wait tăng và query/lock chậm → giảm worker concurrency, không scale thêm worker; xác định query/connection leak trước.

Nếu Redis timeout tăng nhưng Redis CPU/memory/blocked-clients bình thường → coi Redis timeout là triệu chứng và tiếp tục truy dependency/network trước khi scale Redis.

Nếu restart_count tiếp tục tăng → exponential backoff + restart cap; vượt ngưỡng thì stop auto-restart/circuit-break service để điều tra, thay vì restart vô hạn.

Sau phục hồi chỉ có khoảng 2.000 job/phút spare capacity theo giả định đề; replay phải nằm dưới phần dư này và dừng ngay khi backlog tăng ròng hoặc SLO xấu đi.

3. Recovery & Replay Rules

Giảm concurrency của worker gây áp lực DB; giới hạn tổng connection; ưu tiên workload quan trọng. Job đã nhận phải nằm trong durable queue.

Một workload chỉ có một restart manager chính; không để systemd và Docker cùng restart liên tục. Khi shutdown: stop nhận job mới → drain → persist → ACK. Hết drain timeout thì để lease/message redeliver có idempotency.

Chỉ replay sau khi hệ thống ổn định, theo batch nhỏ và nằm trong spare capacity. Dừng replay/tăng tải nếu pool >90%, 5xx >1%, backlog tăng ròng, xuất hiện restart/OOM mới hoặc phát hiện sai dữ liệu.

4. SLO & Alerts

Type	Target	Threshold / baseline
SLO	API availability	≥99,9% valid requests / 30d
SLO	Job completion	≥99% accepted jobs complete within 5 min / 30d
Alert	Backlog	>20.000 and rising for 5 min; compare 12.000 baseline
Alert	DB pool	>90% for 5 min + pool wait rising; baseline target <70%
Alert	5xx/timeout	>1% for 5 min with ≥100 requests; baseline <0,1%

5. Tests & Limitations

```
Lệnh: python -m unittest discover -s tests -v -k q9
```

Kỳ vọng: 1 test, OK.

Giới hạn: phép tính được test offline; các lệnh là runbook read-only và chưa chạy trên production.

Nguồn: PostgreSQL pg_stat_activity [1]; Docker restart policies [2].

Câu 10 — Publishing

CÂU 10 — XUẤT BẢN ĐA NGÔN NGỮ CÓ KIỂM SOÁT

1. Publishing Flow

Stage	Rule
Draft	immutable source_version
Translate	translation_version per language + source_version input
Validate	citation, image rights, prompt-injection checks
Approval	bind approver + exact versions/hash + language/channel
Publish	idempotency/state check before external API
Reconcile	resolve UNKNOWN / external mapping before retry

Bản nguồn, bản dịch, validation và approval đều được lưu theo version. Queue chỉ vận chuyển work; DB giữ action intent, publish status và audit.

2. Version & Idempotency

Mỗi article có source_version bất biến; mỗi bản dịch có translation_version, language, source_version và content_hash. Sửa source hoặc translation tạo version mới, không ghi đè bản đã duyệt.

Publish operation key = tenant_id + article_id + source_version + translation_version + language + channel + action_type. Retry cùng logical operation giữ nguyên key; cùng key nhưng payload khác bị từ chối.

Idempotency DB không bảo đảm provider chỉ tạo một bài; nếu provider không có idempotency contract thì phải lookup/reconcile hoặc manual review.

3. Validation & Approval

Area	What is stored / checked	Block when
Citation	claim, excerpt, source, checked_at, reviewer	source does not support claim/context
Image	asset_id, source, usage-right evidence, attribution, expiry	no valid usage right
Approval	approver, source/translation version, hash, language/channel, manifest	any approved dependency changed
Prompt injection	source content treated as untrusted data	content attempts to grant capability/approval

Approval phải bind với approver, source_version, translation_version, content hash, language/channel và manifest citation/asset. Thay source, translation, image hoặc citation có ảnh hưởng → approval stale.

4. Publish & Reconciliation

Worker kiểm tra approval/idempotency/quyền hiện hành → gọi API ngoài DB transaction → khi có kết quả chắc chắn thì persist PUBLISHED + external_post_id rồi ACK.

Nếu timeout sau khi provider có thể đã tạo bài → UNKNOWN. Reconciler lookup theo operation key/external reference; chỉ retry khi có điều kiện an toàn. “Chưa tìm thấy” trong hệ thống eventual-consistent chưa đủ để kết luận chưa publish.

Dispatch race & external mapping

Approval hợp lệ được dùng để tạo immutable publish_intent chứa exact payload_hash/version. Sau khi intent đã được tạo, source v2 không mutate payload của intent v1; nếu chưa dispatch thì intent cũ có thể bị cancel theo policy, nếu đã dispatch thì phải reconcile kết quả rồi update/retract bằng workflow mới.

Duy trì mapping ổn định (tenant, article_id, language, channel) → external_post_id. Mapping này trả lời “bài nào ngoài provider”, còn operation key trả lời “đây có phải retry của cùng thao tác hay một thao tác mới”.

action_type phải phân biệt CREATE, UPDATE và RETRACT. Version đổi tạo operation mới có chủ đích; retry của cùng operation giữ nguyên idempotency key.

5. Failure Modes

Failure	Recovery	Reconciliation
Source/translation changed after approval	block dispatch; re-translate/validate; require new approval	compare active approval with current versions/hash
Provider created post but response timed out	mark UNKNOWN; lookup before retry	bind external_post_id when matching post found
Validation finds bad citation/image rights/injected instruction	BLOCKED / NEEDS_REVIEW	re-validate and re-approve after correction

6. Tests

Test 1: provider tạo bài nhưng response timeout → UNKNOWN → reconcile → PUBLISHED, không tạo bài thứ hai.

Test 2: source đổi sau approval → STALE_OR_MISSING_APPROVAL, provider không được gọi.

Test 3: translation_version đổi sau approval → approval cũ bị từ chối.

```
Lệnh: python -m unittest discover -s tests -v -k q10
```

Kỳ vọng: 3 tests, OK.

7. Quality & Cost

Quality = tỷ lệ claim/citation được kiểm tra độc lập là đúng và được nguồn hỗ trợ.

Cost = tổng chi phí dịch + LLM validation + retry + xử lý phân bổ / số bản language-channel publish thành công.

Giới hạn: test hiện tại dùng mock provider; chưa kiểm thử API publish thật, ACL thật hoặc chất lượng bản dịch thật.

Nguồn: AWS Builders' Library — Making retries safe with idempotent APIs [1]; OWASP LLM Prompt Injection Prevention Cheat Sheet [2].